# SERVER DEVELOPER PORTFOLIO

김경록 (Roki)

✉ kkl5468@gmail.com

Github : https://github.com/KimGyeongLock

Blog : https://kimgyeonglock.github.io/



## ABOUT ME

김경록

한동대학교 AI·컴퓨터공학심화 전공

GPA 4.17 / 4.5 (상위 8% 이내 졸업)

## WORK




2024.05.~ 2024.12.

Nein to Sick (6인 공동창업)

≪온기≫ 앱 누적 사용자 수 280명

6,000 만원 투자금 유치

## EDUCATION




2025.07.~ 2025.08.

삼성전자 DX 부문 SW 알고리즘 역량 특강 수료



2024.07.~ 2024.12.

카카오테크 부트캠프 풀스택 과정 수료

## SKILL

### Back-End

Java, Spring Boot, JPA / Hibernate

### Database

MySQL, Redis

### Reliability & Performance

k6, JMeter, Prometheus, Grafana, Resilience4j

### Infra

Docker, AWS, Nginx, GitHub Actions

### Frontend

React, Flutter, TypeScript

## 개인 프로젝트
2026.07. ~ 2026.08.



### URL Shortener

백엔드

대규모 시스템 설계
- 01. URL 단축기 구현

프로젝트 정리 블로그

Github Link

## 개인 프로젝트
2025.07. ~ 운영중



### GRADU

프론트엔드 & 백엔드

대학 이수 관리 및
졸업 설계 웹 서비스

GRADU Link

소개페이지

Github Link

## 팀 프로젝트
2023.08. ~ 운영중




### 온기

프론트엔드 & 백엔드

LLM 기반 일상 감정 관리
모바일 어플리케이션

앱스토어

소개페이지

Github Link

## 팀 프로젝트
2024.10. ~ 2024.12.




### 거래함

백엔드

사물함 기반 비대면 중고거래
커머스 웹 서비스

Github Link

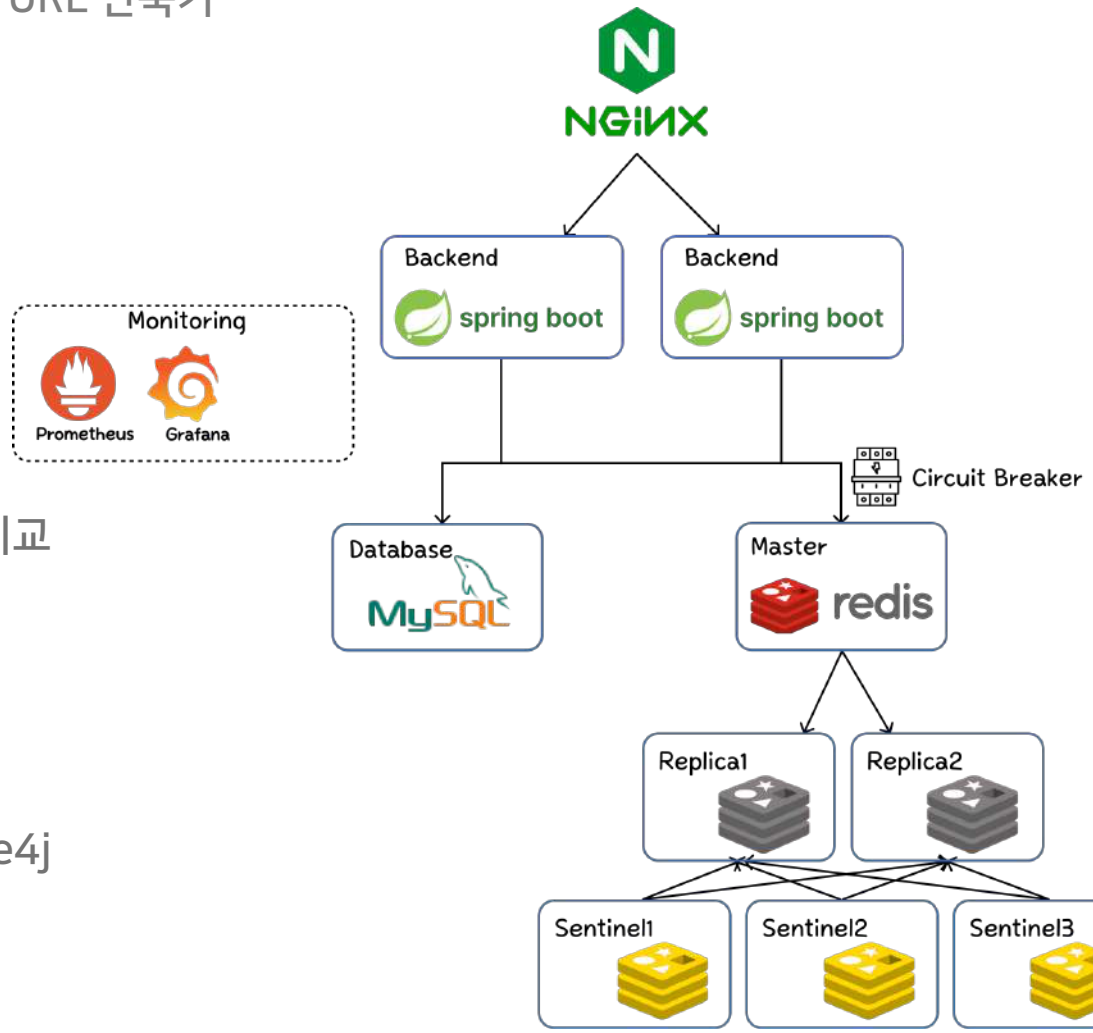


## URL Shortener | 대규모 시스템 설계 및 실험 - 01. URL 단축기

| | |
|---|---|
| 개발 기간 | 2026.07. ~ 2026.08. |
| 개발 인원 | 1명 (개인 프로젝트) |
| 핵심 구현 | 단축 URL 생성 및 302 Redirect |
| | Redis Cache Aside 및 장애 Fallback |
| | App / Redis 다중화 및 Failover |
| | Sequence / Snowflake 기반 ID 생성 전략 비교 |

개발 환경

| | |
|---|---|
| 백엔드 | Java 25, Spring Boot 4.1 |
| DB | MySQL 8.4, Redis 7.4 |
| Infra | Docker, Nginx, Redis Sentinel, Resilience4j |
| 모니터링 | Prometheus, Grafana |
| 부하테스트 | k6 |





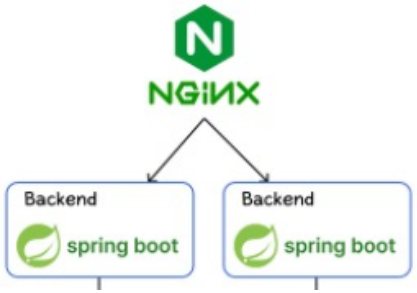
# 단일 장애 지점을 직접 중단해 Failover 구조 검증

## 문제 정의 (App Failover)

단일 App 구조에서는 App 장애가 곧 전체 Redirect 요청 실패로 이어짐

## 설계

App1/App2를 Nginx 뒤에 배치하고 **Active-Active** 구조를 구성
요청 처리 중 한 App에 연결할 수 없으면 다른 App으로 전환




App1 정상 처리
→ App1 직접 중단
→ App2로 요청 전환
→ App1 재기동 후 재합류

## 결과

**446,850 requests → 실패율 0%**

## 문제 정의 (Redis Failover)

Redis 장애 시 MySQL Fallback으로 요청 성공은 유지했지만,
모든 요청이 Redis Timeout을 기다리면서 p95 약 200ms까지 상승

### 1차 개선 (요청 경로 보호)

Circuit Breaker 적용
→ 실패한 Redis 호출 빠르게 차단
→ MySQL 즉시 Fallback

⇨

### 결과

장애 구간 p95
약 200ms → 20~30ms

### 2차 개선 (Redis 자동 복구)

Circuit Breaker는 장애를 우회할 뿐
Redis 자체는 복구하지 못함

Sentinel 적용
→ Master 직접 중단
→ Replica 자동 승격

⇨

### 결과

**938,870 requests**
**→ 실패율 0%**



# App 다중화에 맞춰 ID 생성 방식 재설계

## 초기 구조

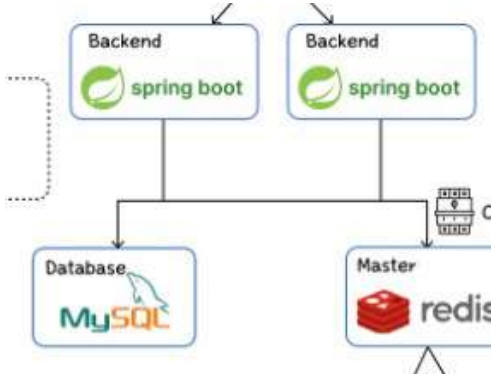
단일 App에서는 DB Sequence가 생성한 ID를 Base62로 인코딩해 Short Code를 생성

장점: 구현 단순, DB의 ID 유일성 보장

Single App → DB Sequence → Base62 → Short Code

## 문제 정의

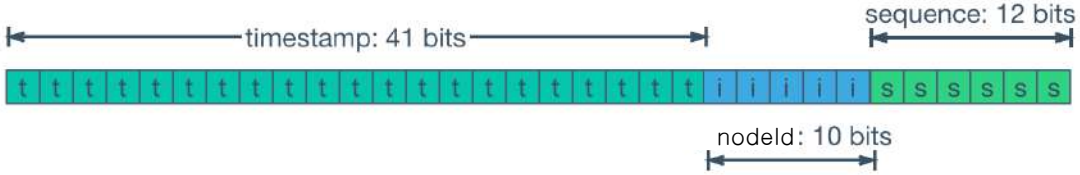
App을 다중화했지만 모든 ID 생성은 여전히 중앙 DB에 의존하는 구조




## 구조 개선

### Snowflake 방식 도입

중앙 DB에 ID 발급을 요청하지 않고도 각 App이 충돌 없이 ID를 생성




64-bit ID를 Timestamp + Node ID + Sequence로 구성하고
App1과 App2에 서로 다른 Node ID를 부여
생성된 ID는 기존과 동일하게 Base62로 인코딩해 Short Code로 사용

Multi Apps → Snowflake → Base62 → Short Code

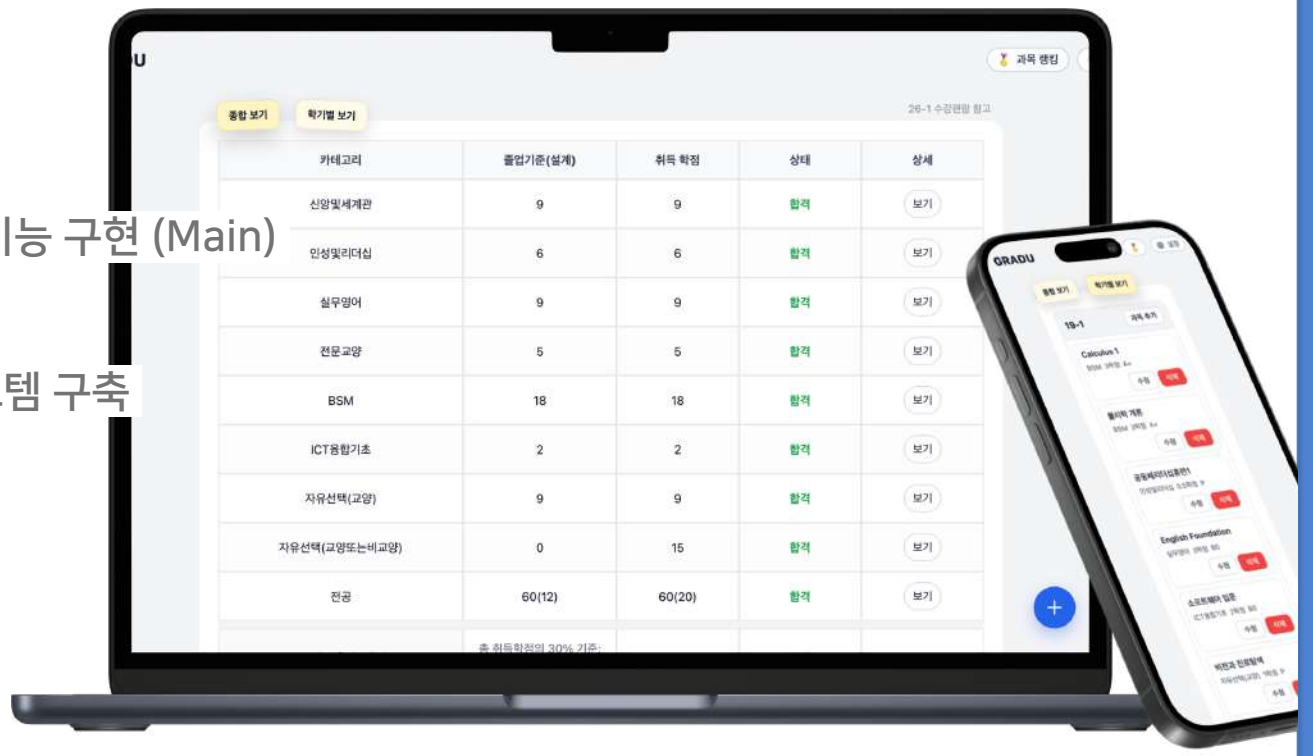
**결과 : 기존과 유사한 성능을 유지하면서 각 App이 독립적으로 ID를 생성하는 분산 ID 생성 구조 검증**

**Sequence: RPS 1,624 / p95 22.23ms**

**Snowflake: RPS 1,660 / p95 21.52ms**

# GRADU | 대학 이수 관리 및 졸업 설계 웹 서비스

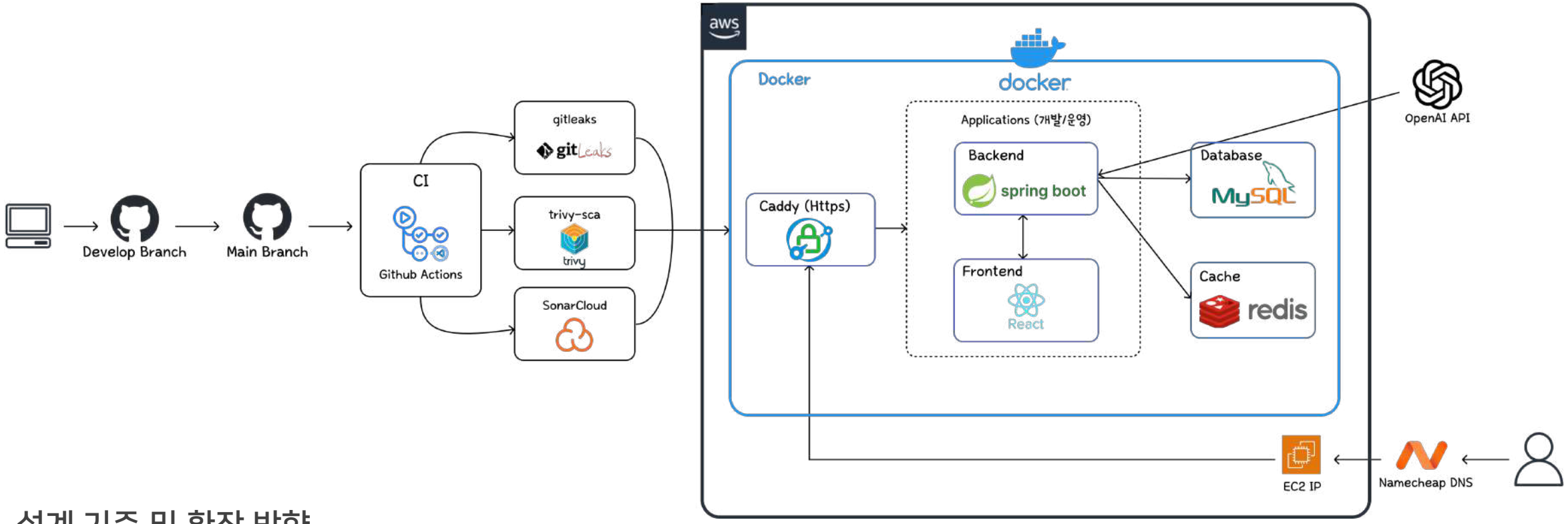
| | |
|---|---|
| 개발 기간 | 2025.07. ~ 운영중 |
| 플랫폼 | Web |
| 개발 인원 | 1명 (개인 프로젝트) |
| 주요 기능 | 수강 계획 설계 및 졸업 가능 여부 진단 기능 구현 (Main) |
| | LLM 기반 과목 일괄 업로드 기능 |
| | 개인 정보 유출 방지 및 과목별 랭킹 시스템 구축 |




## 개발 환경

| | |
|---|---|
| 백엔드 | Spring Boot 3.5.3, Java 21 |
| 프론트엔드 | React 19.1.1 |
| DB | MySQL 8.0, Redis, Spring Data JPA |
| Infra | AWS (EC2, VPC), Docker, Caddy |
| 부하테스트 | k6, JMeter |

## 아키텍처




### 설계 기준 및 확장 방향

초기 사용자 규모와 운영 비용을 고려해 단일 EC2 기반 Docker Compose 구조 적용
Caddy, 애플리케이션, MySQL, Redis를 컨테이너로 분리해 배포 및 관리 단순화
트래픽 증가 시 DB는 RDS, Redis는 ElastiCache로 분리하고 다중 EC2 및 로드밸런서 구조로 확장 가능



# 랭킹 시스템 p95 98% 개선

## 문제 정의

조회 요청마다 DB 집계(COUNT + GROUP BY + ORDER BY)를 반복해 트래픽 증가 시 지연과 DB 부하가 급증
랭킹 API는 기능적으로는 정상(성공률 100%)이지만, 조회 시점 집계 방식 때문에 **동시 요청 증가 시 응답 지연이 급격히 상승**하는 성능 병목 발생

```
@Transactional(readOnly = true)
    public RankingResponseDto getCourseRanking() {
        var major = loadMajorByTerms();

        var liberal = new LiberalRankingDto(
                loadTop10Merged(LIB_FAITH, true),
                loadTop10Merged(LIB_GENERAL_EDU, true),
                loadTop10Merged(LIB_BSM, true),
                loadTop10Merged(LIB_FREE_ELECTIVE, true)
        );

        return new RankingResponseDto(major, liberal);
    }
```

## k6 램프업 부하 테스트 및 Slow Query Log 분석

| 구분 | 지표 | 결과 | 해석 |
|---|---|---|---|
| 성공/실패 | 실패율 ( `http_req_failed` ) | **0.00%** | 요청 실패 없이 정상 응답 유지 |
| 응답시간 | 평균 ( `avg` ) | **1.59s** | 전체 응답이 전반적으로 느려짐 |
| 응답시간 | 중앙값 ( `med` ) | **1.48s** | 대부분 요청이 이미 1초 이상 |
| 응답시간 | p90 | **2.84s** | 상위 10%는 3초 근접 |
| 응답시간 | p95 | **2.85s** | 임계치(800ms) 크게 초과 |
| 응답시간 | 최대 ( `max` ) | **4.95s** | 피크 구간에서 큰 지연 스파이크 |
| 처리량 | 요청 수 ( `http_reqs` ) | **3,353** | 4분 테스트 누적 요청 |
| 처리량 | RPS | **14.38 req/s** | 지연 증가로 처리량 제한 |
| 체크 | status 200 | **100% (3353/3353)** | 기능은 정상이나 성능은 비정상 |

- 로컬 고부하(20→30→50 VU)에서 실패율 0%임에도 p95가 **2.85s**까지 상승 (800ms 목표 초과)
- 평균 응답 **1.59s**, 최대 **4.95s**, RPS **14.38 req/s**
- Slow query log에서 집계 쿼리가 반복 실행되며 요청당 약 **120k rows**를 스캔하는 것 확인 → DB 집계 구조 병목



# 랭킹 시스템 p95 98% 개선

## 문제 해결

### Redis 적용

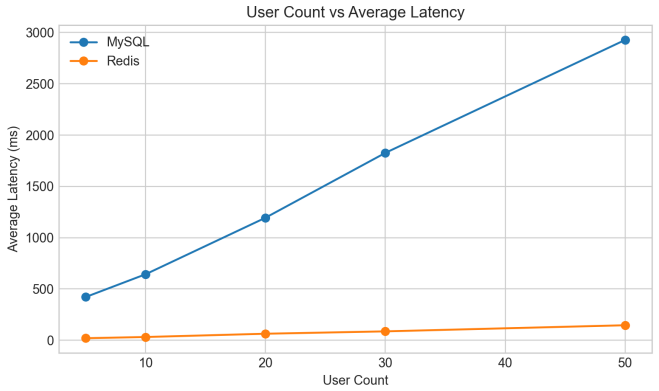
- 조회 시점 DB 집계(COUNT + GROUP BY + ORDER BY) 제거, Redis ZSET 기반 Write-time Aggregation으로 전환
- 카테고리/전공학기별 랭킹 키와 표시명 HASH를 분리해 조회를 ZREVRANGE 중심으로 단순화
- **Redis 우선 조회 + DB fallback**으로 성능/안정성 동시 확보

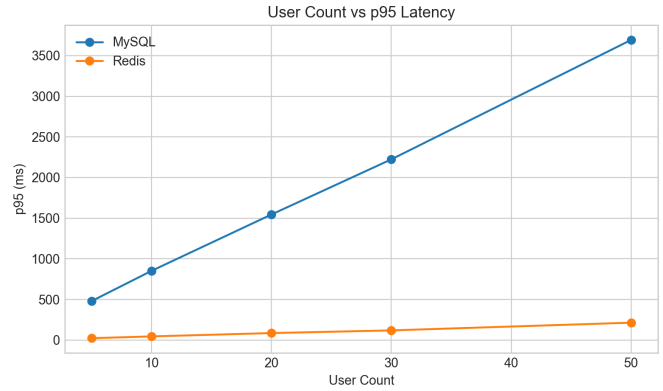
### Lua Script

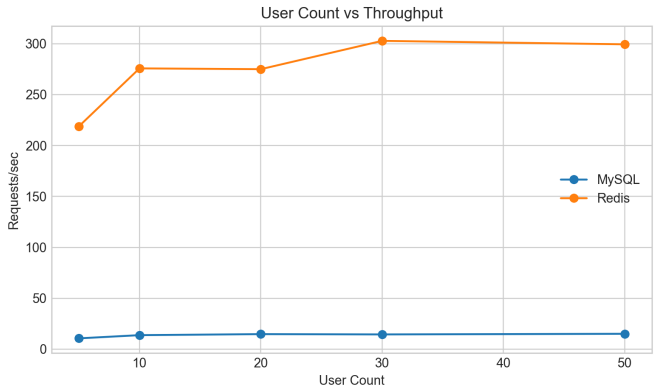
- 다중 키 갱신 중 부분 반영(일부 키만 갱신) 방지, 원자성/정합성 강화
- category ZSET + major ZSET + display HASH 갱신을 Lua 1회 실행으로 통합

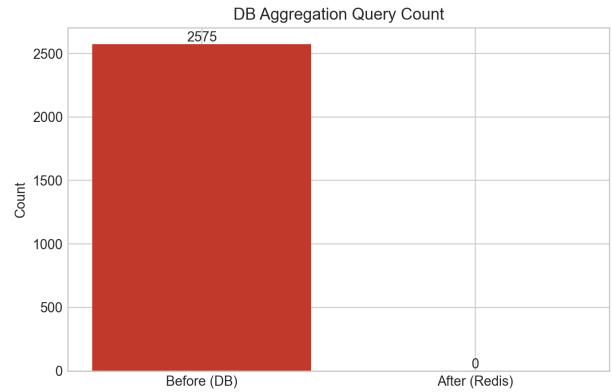
### @Scheduled 배치

- 이벤트 누락·드리프트 자동 복구
- 매일 4시 DB 기준 Redis 랭킹 재빌드 수행




**결과 : Redis 전환 후 동일 부하(20→30→50 VU)에서 p95 2.85s → 47.93ms (98%),**
**RPS 14.38 → 105.45로 성능 병목 해소**
**Slow Query Log 기준 DB 집계 쿼리 2575건 → 0건으로 감소**



# IDOR 취약점 제거

## 문제 정의

학생별 리소스 API에서 경로 ID(studentId) 조작 시 타인 데이터 접근 가능성이 있는 IDOR 취약점 확인

| No | Endpoint | Method | Actor | Target | Expected Behavior | Expected Status | 실제 결과 |
|---|---|---|---|---|---|---|---|
| 1 | /students/{sid}/summary | GET | Me | Me | 본인 졸업요약 조회 가능 | 200 | ✅ |
| 2 | /students/{sid}/summary | GET | Me | Other | 타인 졸업요약 조회 불가 | 403 | ❌✅ |
| 3 | /students/{sid}/summary | GET | Anonymous | Any | 비로그인 사용자는 졸업요약 조회 불가 | 401 | ❌✅ |
| 4 | /students/{id}/courses/all | GET | Me | Me | 본인 전체 과목 목록 조회 가능 | 200 | ✅ |
| 5 | /students/{id}/courses/all | GET | Me | Other | 타인 전체 과목 목록 조회 불가 | 403 | ❌✅ |
| 6 | /students/{id}/courses/all | GET | Anonymous | Any | 비로그인 사용자는 과목 목록 조회 불가 | 401 | ❌✅ |
| 7 | /students/{id}/c | GET | Me | Me | 본인 카테고리별 | 200 | ✅ |

OpenAPI 기준 10개 엔드포인트를 식별하고, 본인/타인/미인증 3유저 상태로 총 30개 인가 테스트 설계
=> 기대 보안 응답(401/403)을 충족하지 못해 인가 정책 일관성 부족 확인

## 문제 해결

1. @CheckStudentAccess + AOP 적용
   JWT studentId vs PathVariable sid 비교, 불일치 즉시 차단
   인가 테스트를 JUnit 통합 테스트로 자동화해 회귀 방지(30 케이스 고정 검증)

2. DB 개인정보 최소화 리팩토링
   **학번/실명 제거**, email_otp는 **Redis TTL**로 이전, 필수 이메일은 **AES 암호화**

## 성과

**인가 테스트 30/30 통과, IDOR 우회 시나리오 차단**
**응답 정책 정리: 미인증 401 / 인증됐지만 권한 없음 403**
**민감 컬럼 약 70% 감소, 직접 식별자 저장 범위 축소로 개인정보 노출 위험 완화**




# 거래함 | 사물함 기반 비대면 중고거래 커머스 웹 서비스

| | |
|---|---|
| 개발 기간 | 2024.10. ~ 2024.12. |
| 플랫폼 | Web |
| 개발 인원 | 3명 (Pair Programming) |
| 담당 역할 | 판매/구매/조회 기능, 알림 및 검색 기능 구현 (Main)<br>구매로직 동시성 제어 및 소셜 데이터 Redis 오프로딩<br>N+1 문제 해결<br>Spring Oauth2 Client 기반 로그인 |

개발 환경

| | |
|---|---|
| 백엔드 | Spring Boot 3.3.4, Java 17 |
| DB | MySQL, Redis, Spring Data JPA |
| 인증/보안 | Spring Security, OAuth2(Naver/Kakao), JWT |
| API/라이브러리 | Swagger(OpenAPI), Validation, Lombok |

| 주차 | 기능 | 내용 |
|---|---|---|
| 1주차 | 로그인 / 로그아웃 | 로그인 구현 |
| | | 로그아웃 구현 |
| | | Refresh Token → 서버 + 쿠키 저장 |
| | | Access Token → Local Storage 저장 |
| 2주차 | 사물함 조회 | 사물함 조회 API |
| | 동시성 | 동시성 이슈 학습 |
| 3주차 | 물품 판매 | 중고 물품 판매 등록 |
| | | 판매 내역 조회 |
| | 메인페이지 | 메인페이지 물품 목록 조회 |
| 4주차 | 물품 구매 | 중고 물품 구매 |
| | | 구매 내역 조회 |
| | 동시성 | 동시성 이슈 해결 |
| 5주차 | 알림 | 판매자 사물함 ID / 비밀번호 전달 |
| | | 구매자 수령 알림 |
| | 검색 | 물품 검색 기능 |
| 6주차 | 소셜 기능 | 물품 조회 수 기능 |
| | | 물품 좋아요 기능 |
| 7주차 | 테스트 | 단위 테스트 학습 및 적용 |
| 8주차 | 성능 | 성능 테스트 및 최적화 |
| 9주차 | 운영 | 운영 모니터링 및 성능 최적화 |
| 10주차 | 운영 | 운영 모니터링 및 성능 최적화 |
| 11주차 | 운영 | 운영 모니터링 및 성능 최적화 |
| 12주차 | 운영 | 운영 모니터링 및 성능 최적화 |




# Race Condition 문제 해결

## 문제 정의

Unit Test를 진행하여 멀티 스레드 환경에서 구매 로직을 진행
100개의 구매 요청을 32개 스레드에서 동시에 실행
예상 재고는 0개였으나 실제 재고는 87개로 확인
**동시 요청 간 갱신 값이 덮어써지며 13건만 정상 반영**
공유 데이터 동시 수정으로 인한 Race Condition 발생

```
expected: <0> but was: <87>
Expected :0
Actual   :87
```

## Pessimistic Lock vs Optimistic Lock

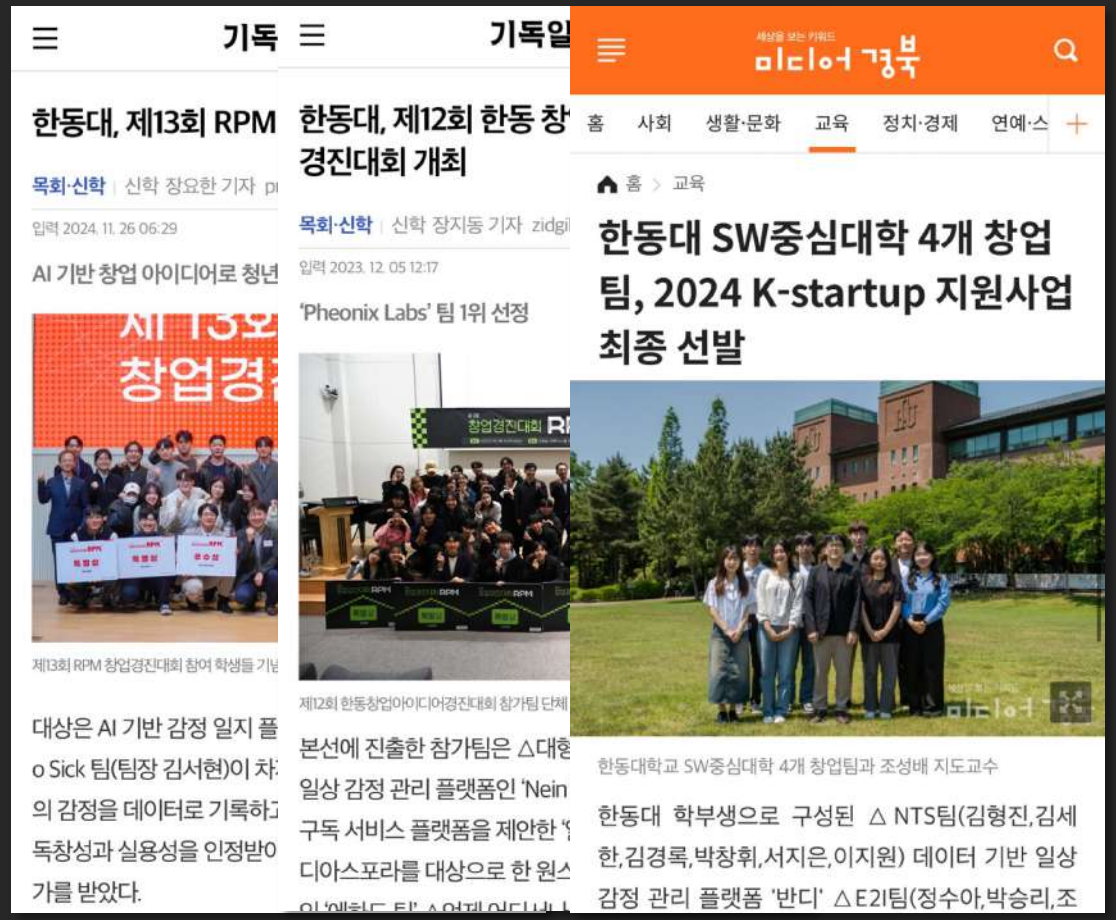
Optimistic Lock 적용 시 충돌 발생 시점마다 재시도 로직이 반복
구매 로직처럼 충돌이 잦은 환경에서는 성능 저하 발생 가능성 확인
Pessimistic Lock을 DB 레벨에 적용해 동시성 이슈 예방

```
2024-11-02T11:53:45.735+09:00 DEBUG 23891 --- [ool-2-thread-10] org.hibernate.SQL
Hibernate: select s1_0.id,s1_0.product_id,s1_0.quantity from stock s1_0 where s1_0.id=? for update
```

stock 테이블에서 특정 id의 재고를 조회하면서 동시에 해당 row에 배타 락 적용

## Pessimistic Lock vs Redis Lock

추후 로그인 단계와 데이터 탐색 최적화를 위해 Redis를 도입
다만 Redis Lock은 DB 트랜잭션과 분리되어 **정합성 문제**가 발생할 수 있어 적용하지 않음




## Awards

제 13회 창업경진대회 RPM 대상
2024-12

SW Festival 프로그래밍 경진대회 장려상
2024-11

캡스톤디자인 경진대회 우수상
2024-06

POSTECH Mini-I-Corps 우수상
2024-02

제 12회 창업경진대회 RPM 장려상
2023-11

SW Festival 스마트어플리케이션 부문 대상
2023-11

SW 창업 경진대회 대상
2023-11

## Certificates

| | |
|---|---|
| SQL 개발자 | 2026-06 |
| 정보처리기사 | 2026-06 |
| 네트워크관리사 2급 | 2022-04 |
| 리눅스마스터 2급 | 2022-04 |
| 컴퓨터활용능력 1급 | 2021-05 |

## Language

| | |
|---|---|
| TOEIC Speaking (IH) | 2025-08 |